

Livro de Programação Competitiva — OBI

Módulo 3 — Técnicas (Prefix Sum, Two Pointers, Binary Search, Greedy)

Estas quatro técnicas resolvem uma fatia enorme dos problemas de nível N1/N2 da OBI sem precisar de estruturas complexas. Domine o "reconhecimento" — a maior parte da dificuldade é perceber qual técnica usar, não implementá-la.

3.A — Prefix Sum (Soma de Prefixo)

1. Introdução

Objetivo: responder somas de intervalos em $O(1)$ após um pré-processamento $O(N)$, em 1D e 2D, e usar Difference Array para atualizações em intervalo.

Pré-requisitos: vetores, loops.

Quando estudar: antes de Two Pointers; é a técnica mais simples do módulo e costuma ser a porta de entrada.

Nível: ★ Fácil (1D) → ★★ Médio (2D, difference array)

2. Intuição

Pense num extrato bancário: se você já sabe o saldo acumulado até cada dia, para saber quanto entrou entre o dia 5 e o dia 10 basta `saldo[10] - saldo[4]` — não precisa somar dia a dia de novo. Prefix sum pré-calcula esses "saldos acumulados" para qualquer vetor.

3. Problema que motivou

Você recebe um vetor de N números e precisa responder Q consultas do tipo "qual a soma dos elementos entre os índices L e R ". Somar na hora para cada consulta é $O(N)$ por consulta. Com prefix sum, cada consulta vira $O(1)$.

4. Construindo a solução

Solução 1 (força bruta): para cada consulta, percorrer de L até R somando $\rightarrow O(N)$ por consulta, $O(N \cdot Q)$ total.

Solução final: construa $\text{prefix}[i] = a[0] + a[1] + \dots + a[i-1]$ uma vez ($O(N)$). Para responder soma de $[L, R]$ (inclusivo), calcule $\text{prefix}[R+1] - \text{prefix}[L] \rightarrow O(1)$ por consulta.

5. Estrutura do algoritmo



```
prefix[0] = 0
para i de 0 até N-1:
    prefix[i+1] = prefix[i] + a[i]

soma(L, R) = prefix[R+1] - prefix[L]
```

6. Implementação manual



```
cpp
vector<long long> a = {3, 1, 4, 1, 5, 9};
int n = a.size();
vector<long long> prefix(n + 1, 0);

for (int i = 0; i < n; i++) {
    prefix[i + 1] = prefix[i] + a[i];
}

// soma do intervalo [L, R] inclusivo:
long long somalIntervalo(int L, int R) {
    return prefix[R + 1] - prefix[L];
}
```

7. Implementação — Prefix Sum 2D (matriz)



```
cpp
```

```

int n, m;
vector<vector<long long>> grid;
vector<vector<long long>> prefix(n + 1, vector<long long>(m + 1, 0));

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        prefix[i+1][j+1] = grid[i][j] + prefix[i][j+1] + prefix[i+1][j] - prefix[i][j];
    }
}

// soma do retângulo com cantos (r1,c1) até (r2,c2), inclusivo:
long long somaRetangulo(int r1, int c1, int r2, int c2) {
    return prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1][c1];
}

```

8. Complexidade

Operação	Complexidade
Construção (1D)	$O(N)$
Construção (2D)	$O(N \cdot M)$
Consulta de soma	$O(1)$

9. Visualizações



a: [3, 1, 4, 1, 5, 9]

prefix: [0, 3, 4, 8, 9, 14, 23]

↑ prefix[i] = soma de a[0..i-1]

soma(2,4) = prefix[5] - prefix[2] = 14 - 4 = 10

= a[2] + a[3] + a[4] = 4 + 1 + 5 = 10 ✓

10. Erros clássicos

Confusão de índices: `prefix` tem tamanho $N+1$, `prefix[i]` é soma de `a[0..i-1]` (exclusivo em `i`)

Esquecer o `+1` de deslocamento ao consultar `prefix[R+1] - prefix[L]`

No 2D, esquecer o termo `- prefix[i][j]` (princípio da inclusão-exclusão)

Usar `int` quando a soma pode estourar (prefira `long long`)

11. Problemas clássicos

Soma de intervalo em vetor estático, múltiplas consultas (fácil)

Soma de submatriz em grid, múltiplas consultas (médio)

Quantidade de subarrays com soma exata igual a K, usando prefix sum + hashmap (médio/difícil)

12. Padrões — quando pensar em Prefix Sum

Se o enunciado diz...	Pense em...
Muitas consultas de "soma entre L e R"	Prefix Sum
"Soma de submatriz", "soma de região"	Prefix Sum 2D
Muitas atualizações de "soma X a todo intervalo [L,R]"	Difference Array
"Quantos subarrays somam exatamente K"	Prefix Sum + hashmap

13. Comparações

Prefix Sum vs Difference Array: Prefix Sum é ótimo para muitas **consultas** de soma com vetor fixo. Difference Array é o inverso: ótimo para muitas **atualizações** em intervalo, com poucas consultas no final.

14. Casos onde NÃO usar

Quando o vetor é atualizado com frequência e você precisa consultar somas a cada atualização — nesse caso prefira Fenwick Tree/BIT (Módulo 4)

Quando as consultas não são de soma de intervalo (ex: máximo de intervalo precisa de Sparse Table ou Segment Tree)

15. Problemas da OBI

Problemas de "soma de região" em matriz — comuns em N1/N2

16. Outros problemas

CSES: "Static Range Sum Queries", "Forest Queries" (2D)

LeetCode: "Range Sum Query - Immutable"

17. Cheatsheet



cpp

// 1D

```
vector<long long> prefix(n+1, 0);
```

```
for (int i=0; i<n; i++) prefix[i+1]=prefix[i]+a[i];
```

```
long long soma = prefix[R+1]-prefix[L];
```

// 2D

```
prefix[i+1][j+1] = grid[i][j]+prefix[i][j+1]+prefix[i+1][j]-prefix[i][j];
```

```
long long soma = prefix[r2+1][c2+1]-prefix[r1][c2+1]-prefix[r2+1][c1]+prefix[r1][c1];
```

18. STL — Difference Array (técnica irmã)



cpp

```
vector<long long> diff(n + 2, 0);
```

// somar V a todo o intervalo [L, R] (inclusivo)

```
void atualizarIntervalo(int L, int R, long long V) {
```

```
    diff[L] += V;
```

```
    diff[R + 1] -= V;
```

```
}
```

// depois de todas as atualizações, reconstrua o vetor final:

```
vector<long long> resultado(n);
```

```
long long acumulado = 0;
```

```
for (int i = 0; i < n; i++) {
```

```
    acumulado += diff[i];
```

```
    resultado[i] = acumulado;
```

```
}
```

19. Template



cpp

```
vector<long long> prefix(n + 1, 0);
```

```
for (int i = 0; i < n; i++) prefix[i+1] = prefix[i] + a[i];
```

```
// consultas: prefix[R+1] - prefix[L]
```

20. Desafios

21. Mini-projeto

Sistema de "extrato bancário": dado um vetor de transações diárias, responda rapidamente "qual foi o saldo acumulado entre o dia X e o dia Y" para várias consultas do usuário.

22. Revisão

- ✓ Por que `prefix` tem tamanho $N+1$?
- ✓ Como generalizar prefix sum para 2D?
- ✓ Quando usar difference array em vez de prefix sum?

23. Flashcards

Q: Fórmula da soma do intervalo $[L,R]$ usando prefix sum? **A:** `prefix[R+1] - prefix[L]`

Q: Quando prefix sum NÃO é a ferramenta certa? **A:** Quando há muitas atualizações no vetor original intercaladas com consultas — nesse caso, use Fenwick Tree.

24. Resumo final

Prefix sum pré-computa somas acumuladas em $O(N)$ (ou $O(N \cdot M)$ em 2D), permitindo consultas de soma de intervalo em $O(1)$. Difference array é o inverso: atualizações de intervalo em $O(1)$, reconstrução final em $O(N)$. Sempre use `long long` para evitar overflow.

25. Checklist de maestria

Conhecimento

- ☐ Sei explicar por que `prefix[i]` é soma de `a[0..i-1]` e não `a[0..i]`
- ☐ Sei quando usar difference array em vez de prefix sum

Implementação

- ☐ Consigo implementar prefix sum 1D e 2D sem erro de índice
- ☐ Consigo implementar difference array

Reconhecimento

- ☐ Reconheço "muitas consultas de soma de intervalo" como prefix sum
- ☐ Reconheço "muitas atualizações de intervalo" como difference array

Competição

☐ Resolvi pelo menos 3 problemas de prefix sum (1D e 2D)

3.B — Two Pointers

1. Introdução

Objetivo: resolver problemas de subarray/substring com janela fixa e variável em $O(N)$, em vez de $O(N^2)$.

Pré-requisitos: vetores, prefix sum (ajuda a entender o ganho de eficiência).

Quando estudar: depois de prefix sum, antes de binary search.

Nível: ★ ★ Médio

2. Intuição

Imagine duas pessoas andando ao longo de uma fila de pessoas, uma começando do início e outra também do início, mas cada uma se move de forma independente conforme uma condição. Em vez de recomeçar do zero a cada tentativa (força bruta $O(N^2)$), os dois ponteiros só andam para frente, nunca voltam — cada um percorre o vetor no máximo uma vez, dando $O(N)$ total.

3. Problema que motivou

Dado um vetor de números positivos, ache o menor subarray contíguo cuja soma é $\geq$ um valor S . Testar todos os subarrays é $O(N^2)$. Com dois ponteiros (janela variável), dá para resolver em $O(N)$.

4. Construindo a solução

Solução 1 (força bruta): para cada início `i`, expanda `j` até a soma atingir S , registre o tamanho $\rightarrow O(N^2)$.

Solução final: mantenha uma janela `[esquerda, direita]`. Expanda `direita` somando; quando a soma da janela atingir/ultrapassar S , tente encolher `esquerda` o máximo possível, atualizando a resposta. Cada ponteiro anda no máximo N vezes $\rightarrow O(N)$.

5. Estrutura do algoritmo



```
esquerda = 0
soma = 0
para direita de 0 até N-1:
    soma += a[direita]
    enquanto soma >= S:
        atualizar resposta com (direita - esquerda + 1)
        soma -= a[esquerda]
        esquerda++
```

6. Implementação manual — janela variável



```
cpp
int menorSubarraySomaMinima(vector<int>& a, int S) {
    int n = a.size();
    int esquerda = 0;
    long long soma = 0;
    int melhor = INT_MAX;

    for (int direita = 0; direita < n; direita++) {
        soma += a[direita];
        while (soma >= S) {
            melhor = min(melhor, direita - esquerda + 1);
            soma -= a[esquerda];
            esquerda++;
        }
    }
    return melhor == INT_MAX ? -1 : melhor;
}
```

7. Implementação — janela fixa (tamanho K)



```
cpp
```



```

long long maiorSomaJanelaFixa(vector<int>& a, int K) {
    int n = a.size();
    long long soma = 0;
    for (int i = 0; i < K; i++) soma += a[i];

    long long melhor = soma;
    for (int i = K; i < n; i++) {
        soma += a[i] - a[i - K]; // desliza a janela: soma um novo, remove o mais antigo
        melhor = max(melhor, soma);
    }
    return melhor;
}

```

8. Complexidade

Variante	Complexidade
Janela fixa	$O(N)$
Janela variável (two pointers clássico)	$O(N)$

9. Visualizações



$a = [2, 1, 5, 1, 3, 2], S = 7$

esq=0 dir=0: soma=2
 esq=0 dir=1: soma=3
 esq=0 dir=2: soma=8 $\geq 7 \rightarrow$ tamanho 3, encolhe:
 soma=6 (removeu $a[0]=2$), esq=1, para de encolher ($6 < 7$)
 esq=1 dir=3: soma=7 $\geq 7 \rightarrow$ tamanho 3
 ...

10. Erros clássicos

Esquecer que a técnica clássica de two pointers **exige que a condição seja monotônica** (ex: só funciona diretamente com valores positivos, onde aumentar a janela só aumenta a soma)

Confundir janela fixa (tamanho constante) com janela variável (tamanho muda conforme a condição)

Não atualizar a resposta no momento certo (antes ou depois de mover o ponteiro, dependendo do problema)

Usar two pointers quando o vetor não está ordenado e o problema depende de ordem (nesse caso, considere ordenar primeiro, se a ordem original não importar)

11. Problemas clássicos

Soma máxima de subarray de tamanho fixo K (fácil)

Menor subarray com soma $\geq S$ (médio)

Maior substring sem caracteres repetidos (médio)

12. Padrões — quando pensar em Two Pointers

Se o enunciado diz...	Pense em...
"Subarray contínuo" / "substring contínua"	Two Pointers / Sliding Window
"Maior/menor janela que satisfaz uma condição"	Janela variável
"Janela de tamanho fixo K"	Janela fixa
Vetor ordenado + par de elementos com soma alvo Two Pointers (ponteiros dos extremos)	

13. Comparações

Two Pointers (extremos) vs Sliding Window: "two pointers nos extremos" é usado em vetor ordenado, com um ponteiro no início e outro no fim, andando um em direção ao outro (ex: par com soma alvo). "Sliding window" é o caso de janela contígua que expande/encolhe (o foco desta seção).

14. Casos onde NÃO usar

Quando a condição não é monotônica (aumentar a janela pode tanto ajudar quanto atrapalhar) — nesse caso, two pointers simples não se aplica diretamente

Quando os elementos podem ser negativos e a soma não é monotônica em relação ao tamanho da janela (jump para prefix sum + estruturas mais avançadas)

15. Problemas da OBI

Problemas de "maior sequência contígua que satisfaz X" — comuns em N1/N2

16. Outros problemas

CSES: "Subarray Sums I/II", "Playlist" (substring sem repetição)

LeetCode: "Longest Substring Without Repeating Characters", "Minimum Size Subarray Sum"

17. Cheatsheet



cpp

// Janela variável

```
int esq = 0; long long soma = 0;
for (int dir = 0; dir < n; dir++) {
    soma += a[dir];
    while (/* condição para encolher */ soma > alvo) {
        soma -= a[esq]; esq++;
    }
    // usar [esq, dir] aqui
}
```

// Janela fixa

```
long long soma = 0;
for (int i = 0; i < K; i++) soma += a[i];
for (int i = K; i < n; i++) soma += a[i] - a[i-K];
```

18. STL — Two Pointers em vetor ordenado (par com soma alvo)



cpp

```
sort(a.begin(), a.end());
int esq = 0, dir = a.size() - 1;
while (esq < dir) {
    long long soma = a[esq] + a[dir];
    if (soma == alvo) { /* achou par */ break; }
    else if (soma < alvo) esq++;
    else dir--;
}
```

19. Template

Ver seção 6 (janela variável) e seção 18 (ponteiros nos extremos) — são os dois templates mais usados.

20. Desafios

Ache a maior substring sem caracteres repetidos.

Ache o menor subarray com soma $\geq S$.

Conte quantos subarrays têm soma exatamente K (combine com prefix sum + hashmap se houver negativos).

Dado um vetor ordenado, ache um par com soma exata a um alvo.

Ache a janela de tamanho fixo K com menor variância (desafio extra).

21. Mini-projeto

Analizador de "sequência de vendas": dado um vetor de vendas diárias, ache automaticamente o menor período consecutivo de dias cuja soma de vendas atinge uma meta.

22. Revisão

- ✓ Por que two pointers dá $O(N)$ em vez de $O(N^2)$?
- ✓ Qual a diferença entre janela fixa e variável?
- ✓ Quando a técnica NÃO se aplica diretamente?

23. Flashcards

Q: Two pointers clássico (janela variável) exige que tipo de condição? **A:** Monotônica — aumentar a janela só pode ajudar ou só atrapalhar, nunca os dois.

Q: Complexidade de two pointers? **A:** $O(N)$, porque cada ponteiro anda no máximo N vezes no total.

24. Resumo final

Two pointers resolve problemas de subarray/substring contínuo em $O(N)$, usando janela fixa (tamanho constante, desliza somando um e removendo outro) ou janela variável (expande e encolhe conforme uma condição monotônica). Também aparece como "ponteiros nos extremos" em vetor ordenado.

25. Checklist de maestria

Conhecimento

- ☐ Sei explicar por que a condição precisa ser monotônica
- ☐ Sei diferenciar janela fixa de janela variável

Implementação

- ☐ Consigo implementar janela fixa e variável sem consultar material
- ☐ Consigo implementar ponteiros nos extremos em vetor ordenado

Reconhecimento

- ☐ Reconheço "subarray/substring contínuo" como sinal de two pointers

Competição

- ☐ Resolvi pelo menos 3 problemas de two pointers/sliding window

3.C — Binary Search

1. Introdução

Objetivo: dominar busca binária em vetor e a técnica mais poderosa do módulo — **busca binária na resposta**.

Pré-requisitos: vetores ordenados, `lower_bound`/`upper_bound` (Módulo 1.C).

Quando estudar: depois de two pointers, é o pilar de muitos problemas de otimização.

Nível: ★ ★ Médio

2. Intuição

Adivinhar um número entre 1 e 100: em vez de tentar 1, 2, 3..., você pergunta "é maior que 50?" e corta o espaço de busca pela metade a cada pergunta. Isso é $O(\log N)$ em vez de $O(N)$.

3. Problema que motivou

Você precisa dividir N livros entre K pessoas, minimizando a maior quantidade de páginas que uma pessoa lê. Não existe fórmula direta, mas existe uma pergunta que dá para responder rápido: "é possível dividir de forma que ninguém leia mais que X páginas?". Essa pergunta é monotônica em X — se funciona para X , funciona para qualquer valor maior. Isso é a receita perfeita para **busca binária na resposta**.

4. Construindo a solução

Solução 1 (força bruta): testar todo valor possível de X , de baixo para cima, verificando se é viável $\rightarrow O(\text{valor máximo} \times \text{custo de verificação})$.

Solução final: binary search sobre X (o valor da resposta), usando uma função `viável(X)` que verifica em $O(N)$ se X é possível $\rightarrow O(\log(\text{valor máximo}) \times N)$.

5. Estrutura do algoritmo



```
busca binária na resposta:  
lo = menor valor possível  
hi = maior valor possível  
enquanto lo < hi:  
    mid = (lo + hi) / 2  
    se viável(mid):  
        hi = mid    // tente um valor menor/melhor  
    senão:  
        lo = mid + 1 // precisa de um valor maior  
resposta = lo
```

6. Implementação manual — busca binária clássica em vetor ordenado



```
cpp  
int buscaBinaria(vector<int>& a, int alvo) {  
    int lo = 0, hi = a.size() - 1;  
    while (lo <= hi) {  
        int mid = lo + (hi - lo) / 2;  
        if (a[mid] == alvo) return mid;  
        else if (a[mid] < alvo) lo = mid + 1;  
        else hi = mid - 1;  
    }  
    return -1; // não encontrado  
}
```

7. Implementação — busca binária na resposta (o padrão mais importante)



```
cpp
```

```

bool viavel(int X, vector<int>& paginas, int K) {
    int pessoas = 1, somaAtual = 0;
    for (int p : paginas) {
        if (p > X) return false; // um livro sozinho já excede X
        if (somaAtual + p > X) {
            pessoas++;
            somaAtual = p;
        } else {
            somaAtual += p;
        }
    }
    return pessoas <= K;
}

int menorMaximo(vector<int>& paginas, int K) {
    int lo = *max_element(paginas.begin(), paginas.end());
    int hi = accumulate(paginas.begin(), paginas.end(), 0);
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (viavel(mid, paginas, K)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

```

8. Complexidade

Operação

Complexidade

Busca binária em vetor $O(\log N)$

Busca binária na resposta $O(\log(\text{intervalo}) \times \text{custo da função viável})$

9. Visualizações



Busca binária na resposta — intervalo de X:

[lo hi]

mid

Se viavel(mid): corta a metade direita (hi = mid)

Senão: corta a metade esquerda (lo = mid+1)

Repete até lo == hi

10. Erros clássicos

Loop infinito por escolher errado entre $hi = mid$ e $hi = mid - 1$ (depende se mid ainda é candidato válido ou não)

Overflow ao calcular $mid = (lo + hi) / 2$ em valores grandes — prefira $mid = lo + (hi - lo) / 2$

Não verificar se a condição de $viavel()$ é realmente monotônica antes de aplicar busca binária na resposta

Confundir os limites iniciais de lo/hi (o intervalo de busca precisa conter a resposta)

11. Problemas clássicos

Buscar um elemento num vetor ordenado (fácil)

Dividir array minimizando o máximo de cada grupo — "livros entre K pessoas" (médio)

Menor tempo para completar uma tarefa com capacidade variável (difícil)

12. Padrões — quando pensar em Binary Search

Se o enunciado diz...	Pense em...
Vetor ordenado + "existe elemento X"	Busca binária clássica
"Minimize o máximo" / "maximize o mínimo"	Busca binária na resposta
A resposta pode ser verificada rapidamente para um valor fixo, e a verificação é monotônica	Busca binária na resposta
"Menor X tal que Y é possível"	Busca binária na resposta

13. Comparações

Busca binária em vetor vs na resposta: a primeira busca um valor **dentro de dados já existentes**; a segunda busca um valor **na faixa de possíveis respostas**, mesmo que esse valor não apareça em nenhum vetor.

14. Casos onde NÃO usar

Quando a condição não é monotônica (não existe um "ponto de corte" único)

Quando `viavel(x)` é cara demais para calcular repetidamente (nesse caso, repense a complexidade total)

15. Problemas da OBI

Problemas de otimização tipo "menor tempo/menor capacidade para atender a demanda" são clássicos de N2/Sênior

16. Outros problemas

CSES: "Factory Machines", "Array Division"

LeetCode: "Koko Eating Bananas", "Split Array Largest Sum"

17. Cheatsheet



```
cpp
// busca binária clássica
int lo=0, hi=n-1;
while (lo<=hi) {
    int mid=lo+(hi-lo)/2;
    if (a[mid]==alvo) return mid;
    else if (a[mid]<alvo) lo=mid+1;
    else hi=mid-1;
}

// busca binária na resposta (minimizando)
int lo = minPossivel, hi = maxPossivel;
while (lo < hi) {
    int mid = lo + (hi-lo)/2;
    if (viavel(mid)) hi = mid;
    else lo = mid+1;
}
// resposta = lo
```

18. STL — `lower_bound`/`upper_bound` como busca binária pronta

Ver Módulo 1.C, seção 7 — `lower_bound`/`upper_bound` já são busca binária da STL, prontos para uso em vetor ordenado.

19. Template

Ver seção 7 (busca binária na resposta) — é o template mais usado em prova de nível N2 em diante.

20. Desafios

Implemente busca binária clássica do zero.

Resolva "dividir livros entre K pessoas minimizando o máximo".

Ache a menor velocidade de leitura para terminar N páginas em D dias.

Ache a raiz quadrada inteira de um número usando busca binária.

Combine busca binária na resposta com uma função `viavel()` que usa greedy internamente.

21. Mini-projeto

Simulador de "alocação de recursos": dado um conjunto de tarefas com custos e um número de "trabalhadores", encontre a menor carga máxima possível usando busca binária na resposta + verificação gulosa.

22. Revisão

- ✓ Qual a diferença entre busca binária em vetor e busca binária na resposta?
- ✓ Como saber se um problema admite busca binária na resposta?
- ✓ Por que `mid = lo + (hi-lo)/2` é mais seguro que `mid = (lo+hi)/2`?

23. Flashcards

Q: Condição necessária para usar busca binária na resposta? **A:** A função de viabilidade precisa ser monotônica em relação ao valor buscado.

Q: Complexidade típica de busca binária na resposta? **A:** $O(\log(\text{intervalo}) \times \text{custo de verificar viabilidade})$.

24. Resumo final

Busca binária corta o espaço de busca pela metade a cada passo, $O(\log N)$. A variação mais poderosa é a busca binária na resposta: quando a pergunta "X é viável?" é monotônica, você busca o menor/maior X viável em vez de testar todos os valores. Cuidado com overflow e com a escolha correta de `hi = mid` vs `hi = mid - 1`.

25. Checklist de maestria

Conhecimento

- ☐ Sei explicar o que torna um problema "buscável por resposta"
- ☐ Sei identificar quando a condição não é monotônica

Implementação

- ☐ Consigo implementar busca binária clássica sem loop infinito
- ☐ Consigo implementar busca binária na resposta com função `viavel()`

Reconhecimento

- ☐ Reconheço "minimize o máximo"/"maximize o mínimo" como busca binária na resposta

Competição

- ☐ Resolvi pelo menos 3 problemas de busca binária na resposta

3.D — Greedy (Algoritmo Guloso)

1. Introdução

Objetivo: reconhecer quando a escolha "ótima local" leva à solução global, aplicar em problemas de intervalos e entender como (informalmente) justificar corretude.

Pré-requisitos: ordenação (Módulo 1.C).

Quando estudar: pode ser estudado em paralelo com busca binária; costuma aparecer combinado com ela.

Nível: ★ ★ Médio (a parte difícil é a prova de corretude, não a implementação)

2. Intuição

Imagine escolher trocos com o menor número de moedas: você pega sempre a maior moeda possível que ainda cabe no valor restante. Greedy é "sempre fazer a escolha que parece melhor agora, sem se preocupar em reconsiderar depois" — funciona quando essa escolha local nunca compromete o resultado global.

3. Problema que motivou

Você tem N atividades, cada uma com horário de início e fim, e uma sala. Quer saber o número máximo de atividades que cabem na sala sem sobreposição. Não dá para testar todas as combinações (exponencial) — mas ordenando por horário de término e sempre escolhendo a próxima atividade compatível, greedy resolve de forma ótima.

4. Construindo a solução

Solução 1 (força bruta): testar todos os subconjuntos de atividades e verificar quais não se sobrepõem $\rightarrow O(2^N)$, inviável.

Solução final: ordene por horário de **término** (não de início!). Percorra em ordem, escolhendo sempre a próxima atividade cujo início é $\geq$ o término da última escolhida $\rightarrow O(N \log N)$.

5. Estrutura do algoritmo



```
ordenar atividades por horário de término
ultimoTermino = -infinito
contador = 0
para cada atividade em ordem:
    se atividade.inicio >= ultimoTermino:
        escolher atividade
        ultimoTermino = atividade.termino
    contador++
```

6. Implementação manual — seleção de atividades (interval scheduling)



cpp

```

struct Atividade { int inicio, termino; };

int maxAtividades(vector<Atividade>& atv) {
    sort(atv.begin(), atv.end(), [](const Atividade& a, const Atividade& b) {
        return a.termino < b.termino;
    });

    int ultimoTermino = INT_MIN;
    int contador = 0;
    for (auto& a : atv) {
        if (a.inicio >= ultimoTermino) {
            contador++;
            ultimoTermino = a.termino;
        }
    }
    return contador;
}

```

7. Implementação — troco com moedas (greedy só funciona com sistema "canônico")

```

□
cpp
vector<int> moedas = {25, 10, 5, 1}; // decrescente

int quantidadeMoedas(int valor) {
    int qtd = 0;
    for (int m : moedas) {
        qtd += valor / m;
        valor %= m;
    }
    return qtd;
}

// ATENÇÃO: greedy de troco só é garantidamente ótimo para sistemas de moedas
// "canônicos" (como o real brasileiro). Para sistemas arbitrários, use DP (Módulo 6).

```

8. Complexidade

Operação	Complexidade
Greedy com ordenação prévia	$O(N \log N)$

Operação

Complexidade

Greedy sem ordenação necessária $O(N)$

9. Visualizações



Atividades (início, término), ordenadas por término:

(1,3) (2,4) (3,5) (6,8)

Escolhe (1,3) $\rightarrow$ ultimoTermino=3

(2,4): início 2 < 3 $\rightarrow$ rejeita

(3,5): início 3 $\geq$ 3 $\rightarrow$ escolhe, ultimoTermino=5

(6,8): início 6 $\geq$ 5 $\rightarrow$ escolhe

Total: 3 atividades

10. Erros clássicos

Ordenar pelo critério errado (por início, em vez de término, no problema de seleção de atividades)

Aplicar greedy em problema onde a escolha local **não** garante o ótimo global sem verificar isso antes (ex: troco de moedas com sistema não-canônico)

Não considerar empates no critério de ordenação (pode precisar de critério de desempate)

Confundir "greedy funciona sempre" com "greedy é intuitivo" — a intuição pode enganar; sempre valide com um contraexemplo mental antes de confiar

11. Problemas clássicos

Troco de moedas com sistema canônico (fácil)

Seleção de atividades / intervalos sem sobreposição (médio)

Cobertura mínima de intervalos (médio/difícil)

12. Padrões — quando pensar em Greedy

Se o enunciado diz...

Pense em...

"Máximo número de intervalos/atividades sem sobreposição"

Greedy — ordenar por término

Se o enunciado diz...

Pense em...

"Menor número de recursos para cobrir tudo"

Greedy — ordenar por início ou término

"Escolha em cada passo a melhor opção local" e o problema tem estrutura de "trade-off simples"

Greedy

Problema parece Greedy mas a escolha local tem exceções

Considere DP em vez disso

13. Comparações

Greedy vs DP: greedy toma uma decisão e nunca reconsidera; DP explora (implicitamente) várias decisões e guarda o melhor resultado de cada subproblema. Greedy é mais rápido, mas só funciona quando o problema tem a "propriedade de escolha gulosa" (a decisão local ótima faz parte de alguma solução global ótima).

14. Casos onde NÃO usar

Quando escolhas locais podem "fechar portas" que seriam necessárias mais tarde (esse é o sinal clássico de que você precisa de DP, não greedy)

Sistemas de moedas não-canônicos para o problema de troco mínimo

15. Problemas da OBI

Problemas de "agendamento"/"intervalos"/"cobertura" — comuns em N1/N2

16. Outros problemas

CSES: "Tasks and Deadlines", "Movie Festival" (seleção de atividades)

LeetCode: "Jump Game", "Gas Station"

17. Cheatsheet



cpp

// Seleção de atividades

```
sort(atv.begin(), atv.end(), [](auto&a, auto&b){ return a.termino < b.termino; });
```

```
int ultimoTermino = INT_MIN, contador = 0;
```

```
for (auto& a : atv) {
```

```
    if (a.inicio >= ultimoTermino) { contador++; ultimoTermino = a.termino; }
```

```
}
```

18. STL — greedy com priority_queue (ex: unir cordas com custo mínimo)



```
cpp
// Problema clássico: unir N cordas de tamanhos diferentes,
// custo de unir duas = soma dos tamanhos, minimizar custo total
priority_queue<int, vector<int>, greater<int>> pq(cordas.begin(), cordas.end());
long long custoTotal = 0;
while (pq.size() > 1) {
    int a = pq.top(); pq.pop();
    int b = pq.top(); pq.pop();
    custoTotal += a + b;
    pq.push(a + b);
}
```

19. Template

Ver seção 6 — o padrão "ordenar + percorrer escolhendo greedy" cobre a maioria dos problemas de nível OBI.

20. Desafios

Resolva seleção de atividades (máximo de atividades sem sobreposição).

Resolva cobertura mínima de pontos com intervalos.

Implemente o problema de "unir cordas com custo mínimo" usando priority_queue.

Ache um contraexemplo onde greedy de troco falha (sistema de moedas não-canônico).

Resolva "máximo de reuniões numa sala" com múltiplas salas disponíveis.

21. Mini-projeto

Sistema de agendamento de sala de reunião: dado uma lista de pedidos de reunião com horário de início/fim, calcule o número máximo de reuniões que podem ser aceitas sem conflito e liste quais foram aceitas.

22. Revisão

- ✓ Por que ordenar por término (e não por início) resolve seleção de atividades?
- ✓ Como reconhecer que greedy NÃO vai funcionar num problema?
- ✓ Qual é a relação entre greedy e DP?

23. Flashcards

Q: No problema de seleção de atividades, por que ordenamos por término? **A:** Porque terminar mais cedo deixa mais tempo livre para futuras atividades — é a escolha que "fecha menos portas".

Q: Greedy sempre funciona? **A:** Não — só quando o problema tem a propriedade de que a escolha local ótima faz parte de alguma solução global ótima. Sempre valide antes de confiar.

24. Resumo final

Greedy faz a escolha que parece melhor a cada passo, sem reconsiderar. Funciona muito bem em problemas de intervalos (ordenar por término) e problemas de "juntar com custo mínimo" (priority_queue). Cuidado: nem todo problema tem estrutura gulosa — quando a escolha local pode prejudicar o futuro, o problema provavelmente pede DP.

25. Checklist de maestria

Conhecimento

- ☐ Sei explicar por que greedy funciona no problema de seleção de atividades
- ☐ Sei identificar quando greedy NÃO é a ferramenta certa

Implementação

- ☐ Consigo implementar seleção de atividades do zero
- ☐ Consigo implementar greedy com priority_queue (ex: unir cordas)

Reconhecimento

- ☐ Reconheço "máximo de intervalos sem sobreposição" como greedy por término
- ☐ Sei diferenciar quando um problema "parece greedy" mas na verdade precisa de DP

Competição

- ☐ Resolvi pelo menos 3 problemas de greedy (incluindo pelo menos 1 de intervalos)

Mapa Mental de Reconhecimento — Módulo 3 (referência rápida)

Se o enunciado diz...

Muitas consultas de soma de intervalo

Pense em...

Prefix Sum

Se o enunciado diz...**Pense em...**

Muitas atualizações de intervalo, poucas consultas	Difference Array
Subarray/substring contínuo	Two Pointers
Vetor ordenado + par com soma alvo	Two Pointers (extremos)
"Existe elemento X" em vetor ordenado	Busca Binária clássica
"Minimize o máximo" / "maximize o mínimo"	Busca Binária na Resposta
"Máximo de intervalos sem sobreposição"	Greedy (ordenar por término)
"Juntar elementos com custo mínimo"	Greedy + Priority Queue